

WEEK 2: PRODUCT AND CATEGORY MANAGEMENT SYSTEM

DEEPIKAA S

Scenario

The e-commerce company wants to maintain information about different products and their categories. Each product belongs to a particular category. The system should store and manage details such as Product Name, Price, and Stock Availability.

1. Design Product and Category Tables

Category Table

The Category table is used to store information about different product categories. The Category_ID uniquely identifies each category.

```
SQL> CREATE TABLE Category1(Category_ID VARCHAR2(7),Category_Name VARCHAR2(20),Description VARCHAR2(50));  
Table created.
```

Product Table

The Product table is used to store details about the products. It contains information such as Product_ID, Product_Name, Price, Stock, and Category_ID.

```
SQL> CREATE TABLE Product1(Product_ID VARCHAR2(7),Product_Name VARCHAR2(20),Category_ID VARCHAR2(7),Price NUMBER(9,2),Stock_Quantity  
Number);  
Table created.
```

The two tables, Category and Product, have been created. Now, we need to define the relationship between the tables and apply the required constraints.

2. Define Relationships and Constraints

Relationship Between Category and Product

A category can contain many products, but each product belongs to only one category.

Therefore, the relationship between the two tables is:

One Category → Many Products

Relationship: One-to-Many (1 :)

Primary Key

A Primary Key is a column that uniquely identifies each record in a table. It does not allow duplicate values.

- Category_ID is the Primary Key in the Category table.
- Product_ID is the Primary Key in the Product table.

```
SQL> ALTER TABLE Category ADD CONSTRAINT PK_category PRIMARY KEY(category_ID);  
Table altered.
```

```
SQL> ALTER TABLE Product ADD CONSTRAINT PK_product PRIMARY KEY(product_ID);  
Table altered.
```

Foreign Key

A Foreign Key is used to connect two tables.

The Category_ID in the Product table acts as the Foreign Key. It refers to the Category_ID in the Category table.

```
SQL> ALTER TABLE Product1 ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category1(Category_ID);  
Table altered.
```

NOT NULL Constraint

The NOT NULL constraint is used when a column must contain a value. It does not allow the field to be left empty.

```
SQL> ALTER TABLE Category1 MODIFY Category_Name VARCHAR2(100) NOT NULL;  
Table altered.  
  
SQL> ALTER TABLE Product1 MODIFY Product_Name VARCHAR2(100) NOT NULL;  
Table altered.
```

CHECK Constraint

The CHECK constraint is used to make sure that the value entered satisfies a particular condition.

For example, the product price can be checked to make sure that it is greater than zero.

```
SQL> ALTER TABLE Product1 ADD CONSTRAINT CHK_Price CHECK (PRICE > 0);  
Table altered.  
  
SQL> ALTER TABLE Product1 ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);  
Table altered.
```

UNIQUE Constraint

The UNIQUE constraint is used to make sure that duplicate values are not entered into a column.

```
SQL> ALTER TABLE Category1 ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);  
Table altered.
```

3. Insert Values

Category Table

Values are inserted into the Category table using the INSERT command.

```

SQL> INSERT INTO Category1 VALUES('&Category_ID','&Category_Name','&Description');
Enter value for category_id: SC001
Enter value for category_name: Kanchipuram Silk
Enter value for description: Traditional silk sarees
old 1: INSERT INTO Category1 VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category1 VALUES('SC001','Kanchipuram Silk',' Traditional silk sarees')

1 row created.

SQL> /
Enter value for category_id: SC002
Enter value for category_name: Cotton Sarees
Enter value for description: Comfortable cotton sarees
old 1: INSERT INTO Category1 VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category1 VALUES(' SC002','Cotton Sarees',' Comfortable cotton sarees')

1 row created.

SQL> /
Enter value for category_id: SC003
Enter value for category_name: Designer Sarees
Enter value for description: Modern designer sarees
old 1: INSERT INTO Category1 VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category1 VALUES('SC003','Designer Sarees',' Modern designer sarees')

1 row created.

SQL> /
Enter value for category_id: SC004
Enter value for category_name: Banarasi Sarees
Enter value for description: Elegant Banarasi sarees
old 1: INSERT INTO Category1 VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category1 VALUES('SC004',' Banarasi Sarees','Elegant Banarasi sarees')

1 row created.

```

```

1 row created.

SQL> /
Enter value for category_id: SC005
Enter value for category_name: Georgette Sarees
Enter value for description: Lightweight georgette sarees
old 1: INSERT INTO Category1 VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category1 VALUES(' SC005',' Georgette Sarees','Lightweight georgette sarees')

1 row created.

```

Product Table

Values are inserted into the Product table using the INSERT command.

```

SQL> INSERT INTO Product1 VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity);
Enter value for product_id: P001
Enter value for product_name: Kanchipuram Silk Saree
Enter value for category_id: SC001
Enter value for price: 8500
Enter value for stock_quantity: 10
old 1: INSERT INTO Product1 VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product1 VALUES('P001',' Kanchipuram Silk Saree',' SC001',8500, 10)

1 row created.

SQL> /
Enter value for product_id: P002
Enter value for product_name: cotton saree
Enter value for category_id: SC002
Enter value for price: 1800
Enter value for stock_quantity: 25
old 1: INSERT INTO Product1 VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product1 VALUES('P002','cotton saree','SC002',1800,25)

1 row created.

SQL> /
Enter value for product_id: P003
Enter value for product_name: designer saree
Enter value for category_id: SC003
Enter value for price: 4200
Enter value for stock_quantity: 15
old 1: INSERT INTO Product1 VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product1 VALUES('P003','designer saree','SC003',4200,15)

1 row created.

```

```

SQL> /
Enter value for product_id: P004
Enter value for product_name: BANARASI SAREE
Enter value for category_id: SC004
Enter value for price: 6500
Enter value for stock_quantity: 12
old 1: INSERT INTO Product1 VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product1 VALUES('P004','BANARASI SAREE','SC004',6500,12)

1 row created.

SQL> /
Enter value for product_id: P005
Enter value for product_name: GEOGRETTE SAREE
Enter value for category_id: SC005
Enter value for price: 2800
Enter value for stock_quantity: 20
old 1: INSERT INTO Product1 VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product1 VALUES('P005','GEOGRETTE SAREE','SC005',2800,20)

1 row created.

```

After Insertion – Category Table

```

SQL> INSERT INTO PRODUCT1 VALUES ('P121','Cot','CAT103',19000,7);

1 row created.

```

4. CRUD Operations

CRUD stands for:

- C – Create
- R – Read
- U – Update
- D – Delete

CRUD operations are used to manage the records stored in the database.

Insert Record

A new product can be added to the Product table using the INSERT command.

```

SQL> INSERT INTO Product1 VALUES('P006','Mysore Silk Saree','SC001',5200,8);

1 row created.

```

Update Price

The price of an existing product can be changed using the UPDATE command.

```
SQL> UPDATE Product1 SET Price=5500 WHERE Product_ID='P006';

1 row updated.
```

Delete a Product

An existing product can be removed from the Product table using the DELETE command.

```
SQL> DELETE FROM Product1 WHERE Product_ID='P006';

1 row deleted.
```

5. Generate Category-wise Product Reports

1. Display all products under each category

This report is used to display all the products belonging to each category.

```
SQL> SELECT C.Category_ID, C.Category_Name, P.Product_ID, P.Product_Name, P.Price, P.Stock_Quantity FROM Category1 C JOIN Product1 P
ON C.Category_ID = P.Category_ID ORDER BY C.Category_ID;
```

CATEGORY_ID	CATEGORY_NAME	PRODUCT_ID	PRODUCT_NAME	PRICE	STOCK_QUANTITY
SC002	Cotton Sarees	P002	Cotton Saree	1800	25
SC001	Kanchipuram Silk	P001	Kanchipuram Silk Saree	8500	10

2. Count the number of products in each category

This report shows how many products are available under each category.

```
SQL> SELECT C.Category_Name, COUNT(P.Product_ID) AS Saree_Count FROM Product1 P JOIN Category1 C ON P.Category_ID = C.Category_ID GRO
UP BY C.Category_Name;
```

CATEGORY_NAME	SAREE_COUNT
Designer Sarees	3
Cotton Sarees	3
Banarasi Sarees	1
Kanchipuram Silk	3
Banarasi Sarees	2
Georgette Sarees	2

6 rows selected.

3. Find the highest-priced product in each category

This report is used to find the product with the highest price in every category.

```
SQL> SELECT C.Category_Name, MAX(P.Price) AS Highest_Price FROM Product1 P JOIN Category1 C ON P.Category_ID = C.Category_ID GROUP BY C.Category_Name;
```

CATEGORY_NAME	HIGHEST_PRICE
Designer Sarees	4200
Cotton Sarees	1800
Banarasi Sarees	6500
Kanchipuram Silk	8500
Banarasi Sarees	6500
Georgette Sarees	2800

6 rows selected.

4. Display categories having more than 5 products

This report displays only those categories that contain more than five products.

```
SQL> SELECT C.Category_Name, COUNT(P.Product_ID) AS Saree_Count FROM Product1 P JOIN Category1 C ON P.Category_ID = C.Category_ID GROUP BY C.Category_Name HAVING COUNT(P.Product_ID) > 5;
```

no rows selected

5. Find the average product price category-wise

This report calculates the average price of products available in each category.

```
SQL> SELECT C.Category_Name, AVG(P.Price) AS Average_Price FROM Product1 P JOIN Category1 C ON P.Category_ID = C.Category_ID GROUP BY C.Category_Name;
```

CATEGORY_NAME	AVERAGE_PRICE
Designer Sarees	4200
Cotton Sarees	1800
Banarasi Sarees	6500
Kanchipuram Silk	8500
Banarasi Sarees	6500
Georgette Sarees	2800

6 rows selected.